



University
of Glasgow

W6-04: Bloom Filters

COMPSCI4064 (H) and COMPSCI5088 (M)

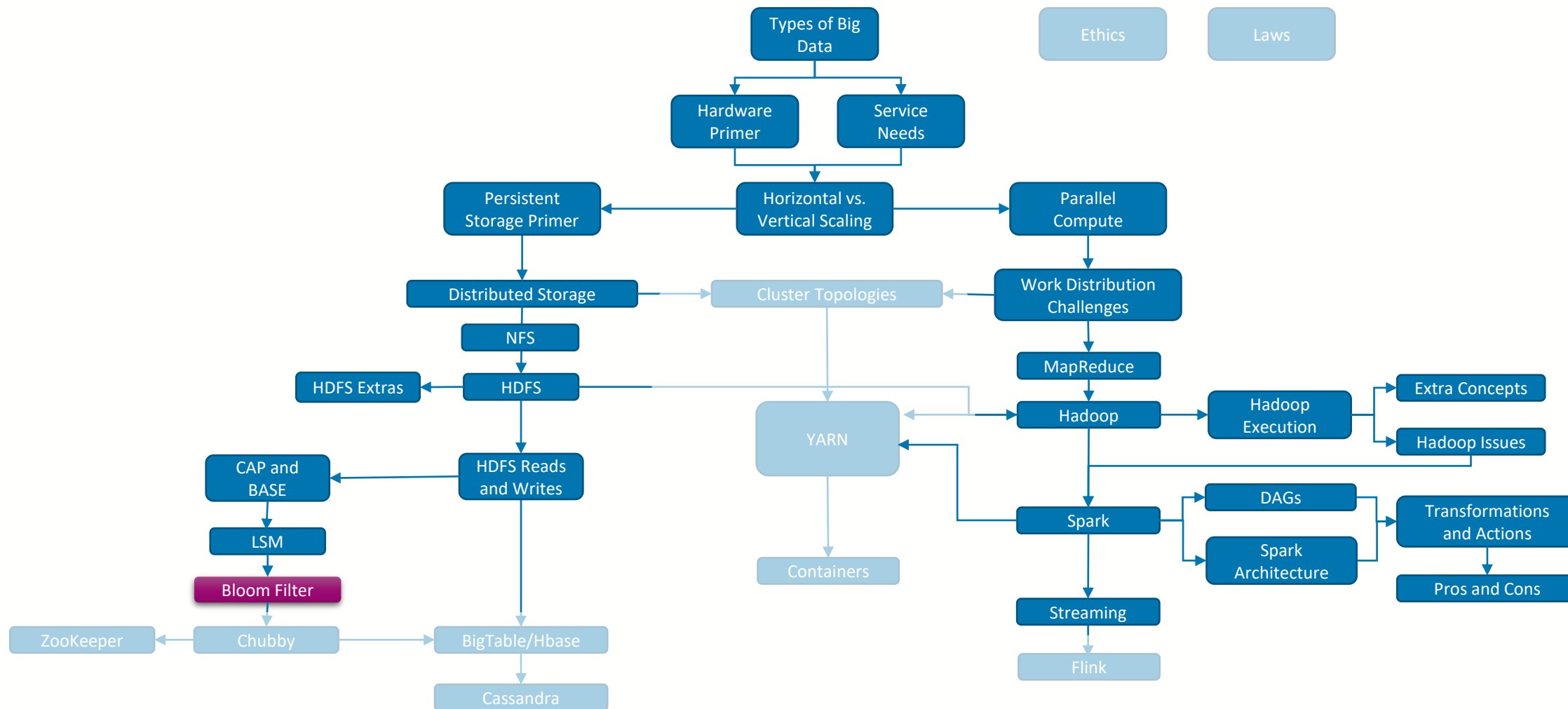
Dr. Richard McCreddie

**WORLD
CHANGING
GLASGOW**

**A WORLD
TOP 100
UNIVERSITY**



Where are we?





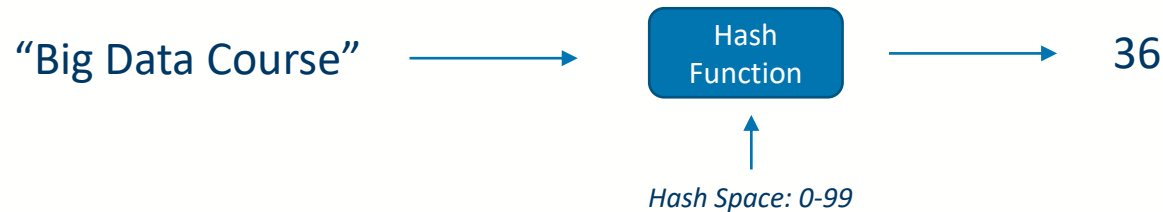
Bloom Filters

- If you read technical documentation for many Big Data storage systems, you will come across **Bloom Filters** being used... so its worth knowing what they are
- Bloom Filters were proposed by Burton Howard Bloom in 1970 and are a **space-efficient way to probabilistically determine whether an item belongs in a set**
 - They are often use to determine whether a **key associated to a storage record (item)** is **held in a file** (a set of items)
- Given a file (a set of items), the bloom filter will tell you either:
 1. The item is **possibly in set**
 2. The item is **not in the set**



Hashing

- The core of Bloom Filters is **Hashing**:
 - The process of **transforming** any given **key** or a string of characters **into another (usually shorter) value**.



- A Hash function takes as **input a key/string** and **outputs a number** or bit array representing that key/string
 - A hashing function will have a hash space, which defines the max-min values that can be output or the size of the bit array respectively
 - Given a input A the hash function will always give out value B
 - Multiple different inputs can result in the same output

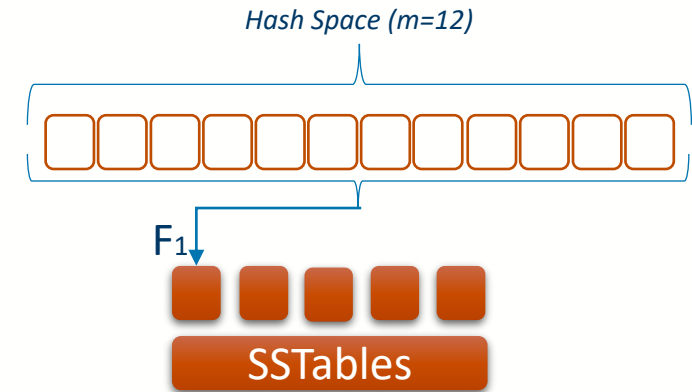


So how does a Bloom Filter work?

- In a log index context, Bloom Filters allow us to look up whether a file contains a record based on its key, lets consider writing records first:

1. Define a Hash Space for our spill file F_1

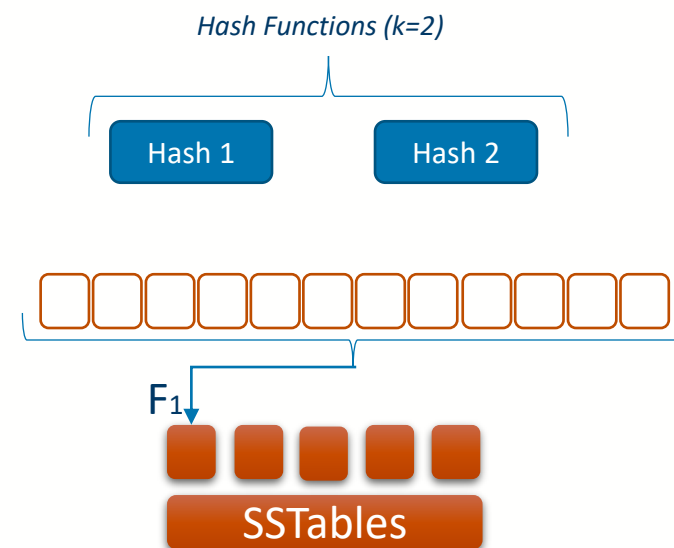
1. A binary array comprised of m bits



So how does a Bloom Filter work?

- In a log index context, Bloom Filters allow us to look up whether a file contains a record based on its key, lets consider writing records first:

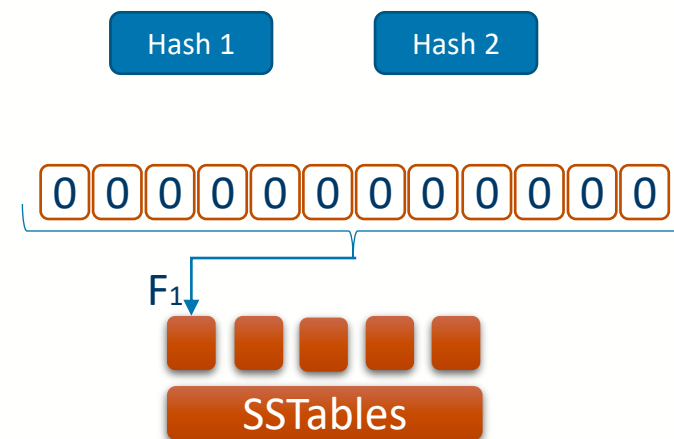
1. Define a Hash Space for our spill file F_1
 1. A binary array comprised of m bits
2. Define k hash functions bounded by m



So how does a Bloom Filter work?

- In a log index context, Bloom Filters allow us to look up whether a file contains a record based on its key, lets consider writing records first:

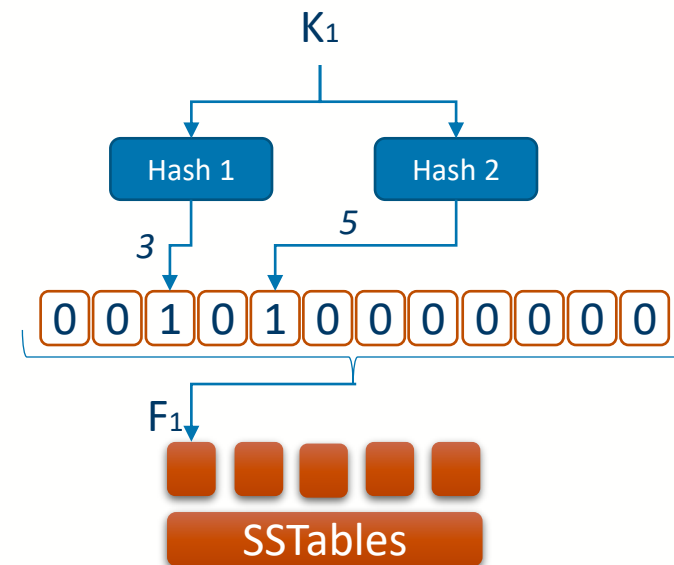
1. Define a Hash Space for our spill file F_1
 1. A binary array comprised of m bits
2. Define k hash functions bounded by m
3. Set all values to 0



So how does a Bloom Filter work?

- In a log index context, Bloom Filters allow us to look up whether a file contains a record based on its key, let's consider writing records first:

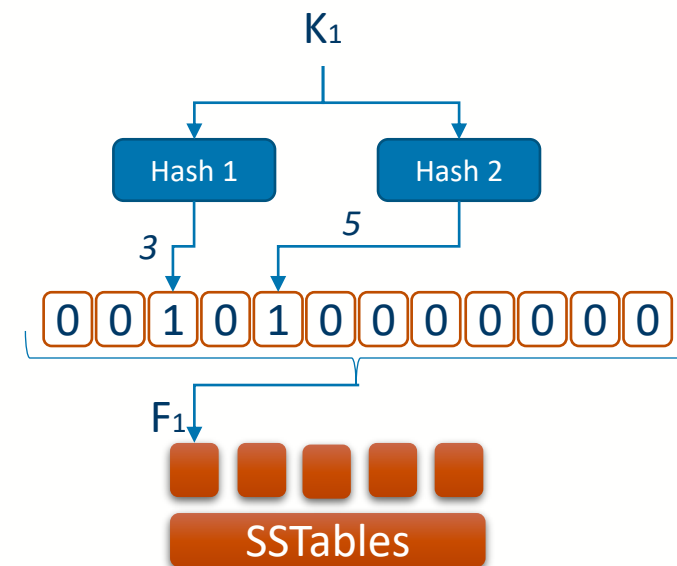
1. Define a Hash Space for our spill file F_1
 1. A binary array comprised of m bits
2. Define k hash functions bounded by m
3. Set all values to 0
4. When we write a record with key to K_1 spill file F_1
 - Hash K_1 using the functions
 - Set the output values to 1 in the hash space



So how does a Bloom Filter work?

- In a log index context, Bloom Filters allow us to look up whether a file contains a record based on its key, let's consider writing records first:

1. Define a Hash Space for our spill file F_1
 1. A binary array comprised of m bits
2. Define k hash functions bounded by m
3. Set all values to 0
4. When we write a record with key to K_1 spill file F_1
 - Hash K_1 using the functions
 - Set the output values to 1 in the hash space



So how does a Bloom Filter work?

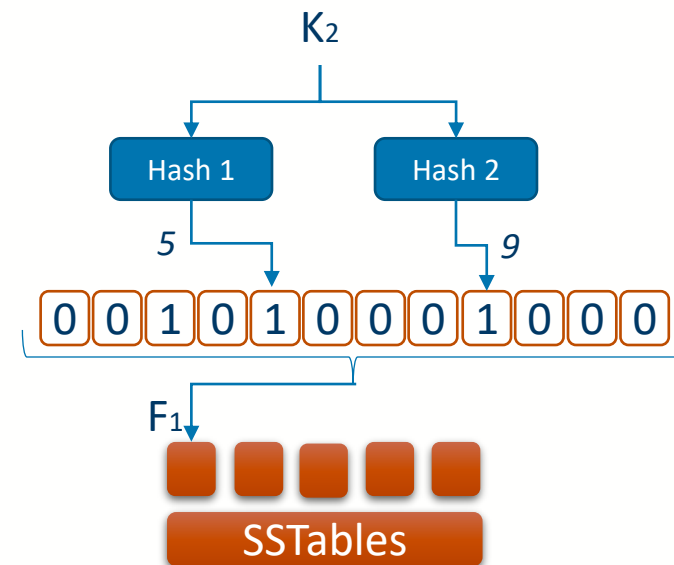
- In a log index context, Bloom Filters allow us to look up whether a file contains a record based on its key, let's consider writing records first:

5. Now we write a second record with key to K_2 spill file F_1

- Hash K_2 using the functions
- Set the output values to 1 in the hash space

6. ... and so on

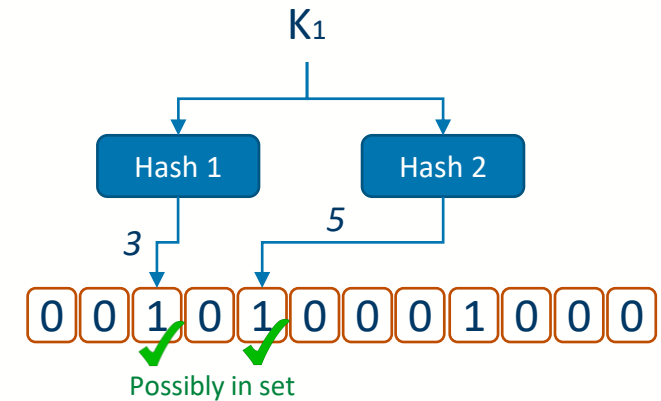
- Note that both K_1 and K_2 mapped to position 5
 - This is a hash collision – where two items receive the same hash value
 - Probability of this occurring decreases as m increases





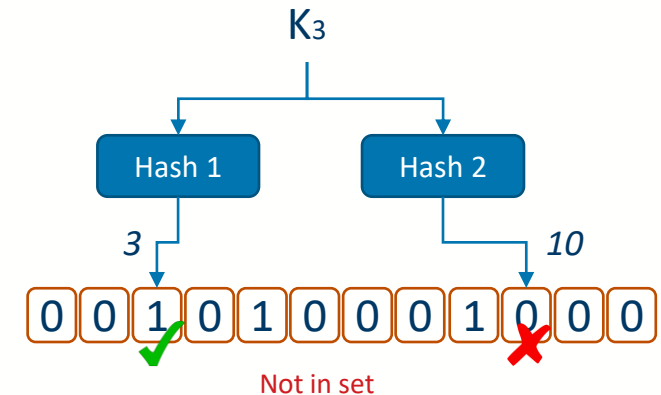
So how does a Bloom Filter work?

- So how do we check whether one of these keys exist in F_1 (read)
- We simply re-hash our key and check all positions
 - If all position are 1 then the key is **possibly in set**
 - If any positions are 0 then the key is **not in the set**



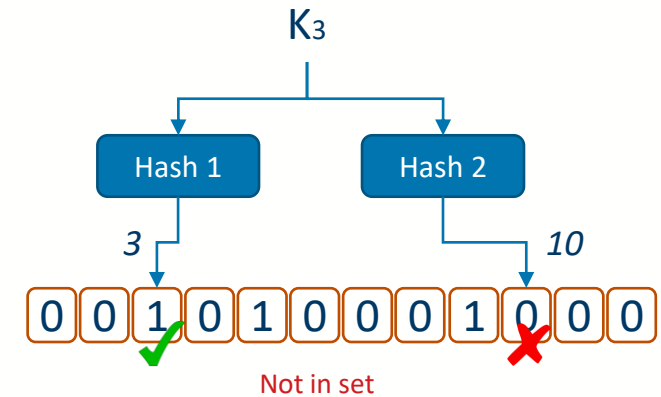
So how does a Bloom Filter work?

- So how do we check whether one of these keys exist in F_1 (read)
- We simply re-hash our key and check all positions
 - If all position are 1 then the key is **possibly in set**
 - If any positions are 0 then the key is **not in the set**



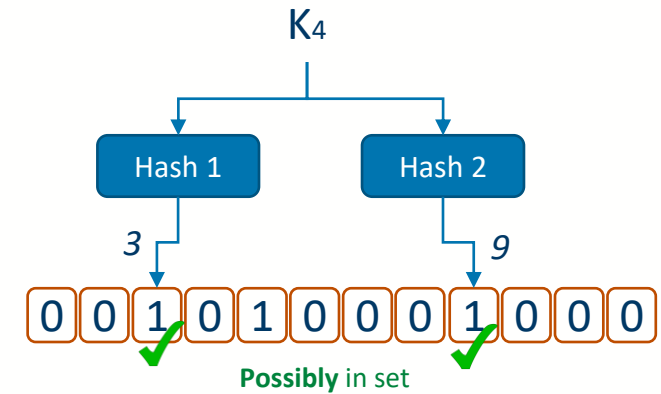
So how does a Bloom Filter work?

- So how do we check whether one of these keys exist in F_1 (read)
- We simply re-hash our key and check all positions
 - If all position are 1 then the key is **possibly in set**
 - If any positions are 0 then the key is **not in the set**
- This works because our has functions guarantee to give the same value for a key



So how does a Bloom Filter work?

- So how do we check whether one of these keys exist in F_1 (read)
- We simply re-hash our key and check all positions
 - If all position are 1 then the key is **possibly in set**
 - If any positions are 0 then the key is **not in the set**
- This works because our has functions guarantee to give the same value for a key
 - However, because of **hash collisions**, just because all positions are set to 1 we cannot be sure that the this is a true hit
 - We can make this less likely by increasing m and/or k





Some Math...

- Probability of false positives:
 - What is the probability that a bit is 0 after adding one element and using just one hash function?
 - What if we use multiple k hash functions?
 - What is the probability that a bit is zero, after we added n elements using k hash functions?
 - The probability of a bit being 1 after adding n elements with k hash functions is:

$$p_1 = 1 - (1 / m)$$

[assume all bits have same prob of being hit]

$$p_2 = (1 - (1 / m))^k$$

$$p_3 = (1 - (1 / m))^{kn}$$

$$= 1 - p_3$$



Some Math...

- What is the probability of a false positive *pfp* after we added *n* elements?

$$pfp = (1 - (1 - (1/m))^{kn})^k \approx (1 - e^{-kn/m})^k$$

[not strictly correct, but is a good approximation]

- Optimal *k*, given *m* and *n*:

$$k_{opt} = (m/n) * \ln(2)$$

- Optimal *m*, given *n*, target *pfp*:

- Optimal *k* is then $k_{opt} = -\log_2(pfp)$

$$m_{opt} = -1.44 * \log_2(pfp) * n$$

- Example Usage: *n*=1000, target *pfp*=5% (0.05)

$$k_{opt} = 5, m_{opt} = 6,224 \text{ bits} = 778 \text{ bytes}$$



University
of Glasgow

Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers



@UofGlasgow